



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



Practical 4 : Singly Linked List: Dynamic Insertion, Deletion, and Traversal

Practical Aim:

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

Problem 1 :

A hospital manages a queue of patient tokens. New critical patients must be added to the front,

routine patients are added to the end, and occasionally a patient with a priority number must be

inserted at a specific position in the queue. Given a sequence of such operations, implement all

three insertion types and print the final queue after each operation.

Describe how your insertion at a specific position works. What does your solution do if the

position given is greater than the current length of the queue? Is that handled explicitly or does it

cause an error?

Problem 2 :

The hospital also needs to remove a patient token from the queue when a patient leaves, print

all remaining tokens from last to first for an end-of-day audit, and display the full queue from

front to back at any point. Implement deletion by value, reverse printing, and forward traversal

on the same queue from Problem 4.1a.

Describe how your reverse printing works. Does it use any extra storage, or does it work purely

by following the links? Can you think of another way to reverse-print without modifying the original queue?

For Code:

<https://github.com/yakshu205/DSA/tree/6abc6fc9b088b332c6c3b366719caf1c414683db/Sorting%20algo/practical-4>

Output:

```
PS C:\Users\yaksh\OneDrive\Desktop\DSA> cd "c:\Users\yaksh\OneDrive\Desktop\DSA\
st_insert_and_delete } ; if ($?) { .\singleList_insert_and_delete }
1-->2-->3-->4-->5-->NULL
*****
1-->2-->3-->4-->5-->NULL
*****
NULL-->5-->4-->3-->2-->1-->
PS C:\Users\yaksh\OneDrive\Desktop\DSA\Sorting algo\practical-4>
```

Key Questions / Analysis / Interpretation to be Evaluated During/After

Implementation :

1. [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something

else entirely? Justify your choice and explain what assumption about the caller your choice makes.

2. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need

to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

Supplementary Problems :

1. A train has n coaches numbered in order. The train manager wants to detach the coach

that is exactly k positions from the rear of the train, without counting from the front. Given

the list of coach numbers and the value k , remove the correct coach and print the remaining sequence.

Describe the approach you used to find the correct coach without knowing the total number of coaches in advance. What happens if k equals the total number of coaches?

2. A dance instructor pairs up students standing in a line and swaps each pair — the first

with the second, the third with the fourth, and so on. Given the list of student IDs, swap every two adjacent students and print the new order. If the total number of students is odd, the last student stays in place.

Describe how your solution handles the swapping. Does it swap the actual node positions or just the values inside the nodes? What is the difference, and does it matter here?

3. A relay race has runners lined up in a chain. The coach wants to identify the runner standing exactly in the middle of the chain to give them a special baton. Given the list of runner IDs, find and print the middle runner's ID. If there are two middle runners, print the second one.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to find the middle in a single pass without knowing the total count first?

4. A film strip has frames stored in a linked sequence. A technician needs to reverse the order of all frames so the film plays backwards. Given the list of frame numbers, reverse the sequence and print the result.

Describe how your solution reverses the links. How many passes through the list does it make, and how many extra nodes does it create? Can you think of a way to do it using

recursion instead?

5. A message is stored as a sequence of characters in a linked chain. A decoder needs to

verify whether the message reads the same forwards and backwards before accepting it.

Given the sequence of characters, determine whether it forms a palindrome and print Yes or No.

Describe the approach you used. Does your solution modify the original sequence at any

point? If yes, does it restore it before finishing, and why might that matter?

Key Skills to be Addressed

Dynamic node insertion and deletion, pointer/link manipulation, sequence traversal and

reversal, two-pointer reasoning, and handling edge cases in dynamic data structures.

Applications :

Task and patient queue management, undo/redo sequences in editors, playlist and media

management, and message validation systems.

Learning Outcome :

Students will be able to implement and manipulate singly linked lists through pointer-based

operations, trace link changes step by step, and reason about the trade-offs between approaches that modify structure versus those that use extra storage.